

BaseballLM: An LLM to SQL Model for Statcast Data

Michael Eisenberg

April 2025

Abstract

The growth of and widespread adaptation of advanced statistics in baseball has led to a revolution in the evaluation and discussion of players. However, the sport has yet to build a publicly available model which can allow people throughout the industry to interact with the data that has made such a profound impact on the game. This research aims to build the first successful LLM model on Statcast data by employing an LLM-to-SQL approach using a pre-trained T5 model which is fine-tuned on human input and SQL query pairs. These findings will shed light on how a successful model can be built and how future work can lead to a usable product that can efficiently answer any question about baseball.

Introduction

The goal of this project was to build a model that generates SQL queries from natural language input to perform baseball analytics using Statcast data. Baseball has long embraced the power of advanced statistics and modeling in many aspects of team building and player evaluation. Many people, however, especially those without technical expertise, lack the ability to navigate large databases and extract insights. These users, from front office executives and scouts to sports media and fans, would greatly benefit from more accessible ways to analyze Statcast data. For example, front office executives and scouts can ask questions such as “Which players had the highest expected batting average against off-speed pitches?” without needing to understand how the data is stored and queried. Similarly, baseball media can quickly pull up various statistics to shape a story and fans can settle debates without any technical knowledge required.

The application of natural language processing (NLP) to translate human input into structured queries will help remove the technical roadblocks. Instead of relying on a team of data analysts to translate an ask into a SQL query, which is not possible for many people and can significantly slow down the time to access the data needed for front office executives and scouts, people can directly interact with the data. This model can promote more informed decision making, a much wider and more streamlined use of the data, and can even help grow the sport with those who lack an understanding of baseball statistics and simply want to learn more about the players and the game. Similar models and approaches already exist in other sectors, like QueryGPT¹ for Uber business intelligence and education via LANTERN² that helps students better understand SQL queries and how to generate them for their purpose. The approach to statcast data, BaseballLM, brings these innovations into the world of baseball, providing the first text-to-SQL model that supports users of all baseball knowledge levels in exploring the game’s most advanced data.

Background

Launched in 2015, Statcast³ revolutionized the collection and use of data in baseball. Statcast began with a combination of camera and radar systems before transitioning to Hawk-Eye in 2020, which uses a set of high-speed cameras around the stadium. These advanced tracking

¹ Johnson et al. “QueryGPT – Natural Language to SQL Using Generative AI”.

² Wang et al. “Towards Enhancing Database Education: Natural Language Generation Meets Query Execution Plans”.

³ “Statcast Glossary”. *MLB.com*, Major League Baseball.

systems generate detailed data on every aspect of the game, including sprint speed, pitch spin, exit velocity, and much more. This technology has provided teams and the general public with the ability to access millions of rows of data for every season, fueling the widespread adoption of advanced statistics. Teams can now optimize their front office strategy and even make decisions on the fly in games while fans and the media can create more engaging narratives and lead more informed discussions. However, accessing and interpreting Statcast data remains challenging for many users due to the large technical expertise required to use it.

In parallel to the advances in Statcast technology, recent advances in natural language processing (NLP) have made it easier to perform semantic parsing, the task of translating language into structured queries. An early approach, PRECISE⁴, was a rule-based system that “for a broad class of *semantically tractable* natural language questions, PRECISE is guaranteed to map each question to the corresponding SQL query”. However, as the paper *Towards a Theory of Natural Language Interfaces to Databases* states, that despite the high correctness at the time compared to other models, PRECISE had some limitations and could not handle certain categories of inputs. Further research yielded Seq2SQL⁵, which relies on reinforcement learning to improve translation to SQL queries, and SQLNet⁶, which relies on a template representation of SQL queries to improve upon reinforcement learning techniques from models such as Seq2SQL and yielding a higher accuracy.

In recent years, with the introduction of large language models (LLMs) such as BERT⁷ and T5⁸, much progress has been made in supporting human interaction and conversation with machines. More recent work has proposed methods such as SGU-SQL⁹, which guides LLMs using structured representations during training, and parameter-efficient fine tuning approaches like LoRa¹⁰ and subsequently DoRa¹¹. These techniques allow for the adaptation of LLMs towards Statcast data without the need to retrain a full model. In the context of this project, they offer intriguing approaches for building an LLM that supports conversational access to Statcast data.

Approach

Data

For the successful implementation of a LLM-to-SQL model on Statcast data, the Statcast data needed to be imported into a PostgreSQL database. This was achieved by setting up a Postgres database on Amazon RDS, a managed relational database service, and via downloading statcast data via the *pybaseball* module. The *pybaseball* library contains 20 relational tables which were used for the building of the models, with data relating to each area of the game. The goal of using the 20 tables was to provide the model with data on each component of the game of baseball, including advanced defensive metrics like Outs Above Average, running splits, batter performance against different pitchers, a pitcher’s ability to induce strikeouts with different pitches, and much more. This data allows for the building of a proof of concept LLM-to-SQL model on Statcast data, from which a more advanced model – and even a chatbot – on a larger baseball dataset can be built.

⁴ Popescu et al. “Towards a Theory of Natural Language Interfaces to Databases”.

⁵ Zhong et al. “Generating Structured Queries from Natural Language using Reinforcement Learning”.

⁶ Xu et al. “Generating Structured Queries From Natural Language Without Reinforcement Learning”.

⁷ Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”.

⁸ Raffel et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”.

⁹ Zhang et al. “Structure Guided Large Language Model for SQL Generation”.

¹⁰ Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”.

¹¹ Liu et al. “DoRA: Weight-Decomposed Low-Rank Adaptation”.

Training Approach

To build the LLM-to-SQL model, training data was generated on each table which was included, where a natural language input was mapped to a corresponding correct SQL query. The natural language input is some question to which a user might want the answer and the SQL query is the correct query to obtain the data from the Postgres database, given the setup.

Due to time and hardware constraints, only human input which mapped to SELECT queries on single tables without joins were included in the training data. A more complex and complete model would necessarily include more advanced queries as well as the creation of indexes on specific features of different tables. Additionally, large statcast tables such as *statcast_batter* and *statcast_pitcher* were not included in the model due to memory and time limitations. Those tables each contain millions of rows and approximately 100 features. While techniques such as sharding and feature augmentation can help improve the ability of the model to generalize to different input pairs, preprocessing and training time would still increase significantly. Overall, SELECT queries were included, as well as queries with different aggregations and on multiple features on a single table.

SQL Query Templating Approach

Although only a small class of SQL queries were used to train the model, an approach was still required to ensure the ability to generalize to unseen natural language input. Suppose the model was trained on specific natural language to SQL query mappings such as the following example:

```
How many barrels did Aaron Judge hit in 2020?
```

```
SELECT barrels FROM batter_exitvelo_barrels WHERE first_name =  
'Aaron' AND last_name = 'Judge' AND year = 2020;
```

Training the LLM-to-SQL model on similar queries such as the above example would require the model to be trained on every possible natural language input to SQL query example that could be generated by a user. Since that method clearly is not scalable nor is it even possible, a different approach must be taken. That approach is a templating approach, where both the human input and corresponding SQL query were created with placeholders for different values. The above training pair relating to Aaron Judge was then reformulated as:

```
How many barrels did [PLAYER] hit in [YEAR]?
```

```
SELECT barrels FROM batter_exitvelo_barrels WHERE first_name =  
'[FIRST]' AND last_name = '[LAST]' AND year = [YEAR];
```

In this approach, training data was templated during the entirety of training while the test natural language input was preprocessed to generate a template SQL query, from which the SQL to execute against the database can be fully constructed. Test metrics such as BLEU Score and component-wise F1-Score were then evaluated against the reconstructed SQL query. By following this approach, the model can learn the expected structure of the SQL queries rather than the exact SQL queries themselves. The generated SQL queries were run against the database for validation of their accuracy but the primary metrics of evaluation are the aforementioned BLEU and F1 scores.

Baseline

For the baseline model, and as a basis for all models created, the pretrained T5 model was the method selected. Other approaches such as BERT were considered but eventually not used due to the design of T5 being built on a sequence to sequence transformer.

To build a baseline model, a pre-trained T5-Small model was trained on the templated natural language to SQL inputs without any parameter efficient tuning methods. This baseline was chosen for the lower training time, and ability to more quickly determine how the model can be improved. Two baseline models were trained, one with three training epochs and another with five training epochs.

Modeling

Modeling the human input to SQL generation continued the use of a pre-trained T5-Small model fine-tuned on the SQL generation training data. Each additional model was an approach to improving upon different results of the additional model while considering new methods to ensure training time did not exceed any reasonable limit.

The following models were built in addition to the baseline. Each of these models used a different learning rate from the baseline, as well as warmup training steps and L2 regularization.

- T5-Small with Regularization and warm-up steps
- T5-Small with LoRa and 3 training epochs
- T5-Small with LoRa and 5 training epochs
- T5-Small with DoRa and 3 training epochs
- T5-Small with DoRa and 5 training epochs

LoRa and DoRa are both parameter efficient tuning methods which learn adaptations to the pretrained model without modification of the original model. Both are used to build an LLM-to-SQL model for statcast data to showcase the ability to limit training time while also maintaining high performance against the test set.

Results

The table below shows the results for each model in terms of the BLEU score and Component-wise F1-score.

Model	Epochs	Wall Time	BLEU Score	Component-Wise F1-Score
T5-Small (Baseline)	3	2h 35 min	0.6808	0.79925
T5-Small (Baseline)	5	3h 55 min	0.75955	0.83928
T5-Small Regularized	3	2h 37 min	0.85374	0.86922
T5-Small with LoRa	3	1h 38 min	0.55812	0.75
T5-Small with LoRa	5	3h 17 min	0.6474	0.79029
T5-Small with DoRa	3	1h 59 min	0.56697	0.759
T5-Small with DoRa	5	3h 16min	0.64797	0.79714

Each of the models which were built yielded a moderately high to high component-wise f1-score between 0.75 and 0.87, while the BLEU scores were more varied. Additionally, as expected, models trained on five epochs performed better in both metrics than their counterparts trained on three epochs. This suggests that, without consideration to training time, that training for more epochs will not lead to a model which overfits on the training data.

It is also important to note which model yielded the best results on the test dataset, the T5-Small Regularized model without any parameter efficient tuning. This model resulted in a much higher BLEU score than every other model as well as the highest component-wise f1-score. Additionally, LoRa and DoRa models both performed worse than models without either. This does not indicate an issue with LoRa or DoRa but rather they likely performed worse because of a suboptimal configuration and high complexity of generating SQL queries for one of many different tables based on natural language input. In general, however, the models require a lot of fine tuning, including the learning rate, regularization, number of epochs and number of warm-up steps. Without these optimizations, the model will struggle to successfully generate the structures of the different SQL queries.

Finally, it is important to consider the wall time for each of the models and how LoRa and DoRa made an impact with respect to the baseline models. LoRa led to an improvement of approximately 37 percent on three epochs of training and approximately 16 percent over 5 epochs. On the other hand, DoRa led to a 23 percent improvement on three epochs of training and a similar improvement as LoRa on five epochs. This level of improvement can be expected due to the overhead of training and relatively small size of a T5-Small model compared to larger LLMs. However, they still provide value in decreasing the training time while maintaining their aforementioned level of performance on the test dataset.

Conclusion

Building an LLM-to-SQL model is challenging, especially for a large dataset such as statcast which involves many millions of rows of data and a large number of tables for which to generate SQL queries. The complexity of the tables and their relationships with each other, along with the limitations of compute resources and time required the building of a simpler LLM-to-SQL model by fine-tuning a pretrained T5-small model on the natural language to SQL query training dataset. However, despite these limitations, it was clearly demonstrated that such a model can generate valid SQL queries, especially given the templating of natural language and SQL queries during training. Each model, even the baselines, yielded a high component-wise f1-score. The highest score was for the model with regularization, warm up steps, a smaller learning rate, and no parameter efficient fine tuning. This component-wise f1-score of 0.86922 shows a strong ability to build the structure of the SQL queries while leaving some room for improvement in future iterations. Additionally, while the parameter efficient fine tuning procedures of LoRa and DoRa did not yield improvements on the test set, they did lead to notable decreases in the time it took to train the model while not having a significantly worse performance in terms of the f1-score. For future work on this task, these could prove useful especially when working with more complex training data, more optimal LoRa and DoRa configurations, and when using a large pretrained T5 model.

These models provide a strong initial foray into generating LLM-to-SQL models for statcast data. In future work, limitations surrounding compute and time resources can be handled, where larger T5 models can be fine-tuned on a more complex database and training corpus, as well as the addition of baseball data from sources other than statcast. Future work can also involve a larger hyperparameter search and schema-aware prompts, among other natural language processing techniques. With this work, a successful model can be built as a backend for a chatbot that is usable by baseball teams at all levels for scouting, baseball media for building compelling stories, and by fans for settling debates.

Appendix I - Sources

- Devlin, Jacob, et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” *arXiv*, 11 Oct. 2018, <https://arxiv.org/abs/1810.04805>.
- Liu, Haokun, et al. “Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning.” *arXiv*, 26 Aug. 2022, <https://arxiv.org/abs/2205.05638>.
- Liu, Shih-Yang, et al. “DoRA: Weight-Decomposed Low-Rank Adaptation.” *arXiv*, 9 Jul. 2024, <https://arxiv.org/abs/2402.09353>.
- Popescu, Ana-Maria, Oren Etzioni, and Henry Kautz. “Towards a Theory of Natural Language Interfaces to Databases.” *Proceedings of the 8th International Conference on Intelligent User Interfaces*, 2003, pp. 149–157. ACM Digital Library, <https://doi.org/10.1145/604045.604070>
- Raffel, Colin, et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.” *arXiv*, 23 Oct. 2019, <https://arxiv.org/abs/1910.10683>.
- “Statcast: Glossary”. <https://www.mlb.com/glossary/statcast>.
- Uber Engineering. “QueryGPT – Natural Language to SQL Using Generative AI.” *Uber Blog*, 19 Sept. 2024, <https://www.uber.com/blog/query-gpt/>.
- Wang, Weiguo, et al. “Towards Enhancing Database Education: Natural Language Generation Meets Query Execution Plans.” *arXiv*, 2021, arxiv.org/abs/2103.00740.
- Xu, Xiaojun, Chang Liu, and Dawn Song. “SQLNet: Generating Structured Queries From Natural Language Without Reinforcement Learning.” *arXiv*, 13 Nov. 2017, <https://arxiv.org/abs/1711.04436>.
- Zhang, Qinggang, et al. “Structure Guided Large Language Model for SQL Generation.” *arXiv*, 19 Feb. 2024, <https://arxiv.org/abs/2402.13284>.
- Zhong, Victor, Caiming Xiong, and Richard Socher. “Seq2SQL: Generating Structured Queries from Natural Language Using Reinforcement Learning.” *arXiv*, 9 Nov. 2017, <https://arxiv.org/abs/1709.00103>.